CS683 Programming Assignment 2

Kunj Bhesaniya (23b0995), Dhruvraj Merchant (23B1041),
Harshul Suthar (23B0968), Dipen Sojitra (23B0913)

1 Task 1: Stride Pattern Prefetching

1.1 Task 1B: SPP design

IP table - same structure as the IP-stride prefetcher's table (fully associative, LRU), enlarged to 128 entries with a 10-bit IP tag.

Uniform mode - Each entry has a 2-bit confidence counter for its last stride: $+1$ when the stride repeats, -1 otherwise. The stored stride is replaced only when the confidence is 0. When confidence ≥ 1, the prefetcher issues `addr + k*stride` for $k = 1..$degree.

Non-uniform mode - Each IP keeps a 7-bit signature of its recent strides:

$$\text{sig} = ((\text{sig} \ll 1) \oplus (\text{stride} \,\&\, \texttt{0x7F})) \,\&\, \texttt{0x7F}$$

The signature indexes a signature map (128 entries, shared by all IPs) that stores the next expected stride and a 2-bit confidence. When the **uniform confidence is 0**, the prefetcher walks the map: predict a stride, update the signature as if the stride happened, and repeat for up to *degree* steps. Steps with confidence 0 advance the walk but do not prefetch.

Page boundary - Strides are limited to $-63..+63$ (one page has 64 lines, so the maximum useful stride is 63). Prefetch targets outside the current page are dropped. A larger stride resets the training of that IP. Prefetches fill into L1 while the L1D MSHR is less than half full, and only into L2 otherwise.

Fill level - As in the IP-stride prefetcher, prefetches are filled into L1D while the L1D MSHR is less than half full and only into L2 otherwise.

Accuracy-based throttling - Every 256 prefetch fills, the accuracy over that epoch is computed as useful prefetches/prefetch fills (using ChampSim's `pf_useful` and `pf_fill`). If accuracy > 0.50 the degree is increased by 1; if accuracy < 0.40 it is decreased by 1; otherwise it is unchanged. The degree stays in 1..6 and starts at 3.

Final parameters

The final values are listed below; the data behind each choice is in Section 1.4.

Parameter	Value
IP table	128 entries, 10-bit IP tag
Signature map	128 entries (7-bit signature)
Uniform confidence threshold	1
Signature confidence threshold	1
Confidence counters	2 bits
Degree range (start)	1–6 (3)
Accuracy thresholds (low / high)	0.40 / 0.50
Epoch length	256 prefetch fills

Table 1: Final SPP parameters.

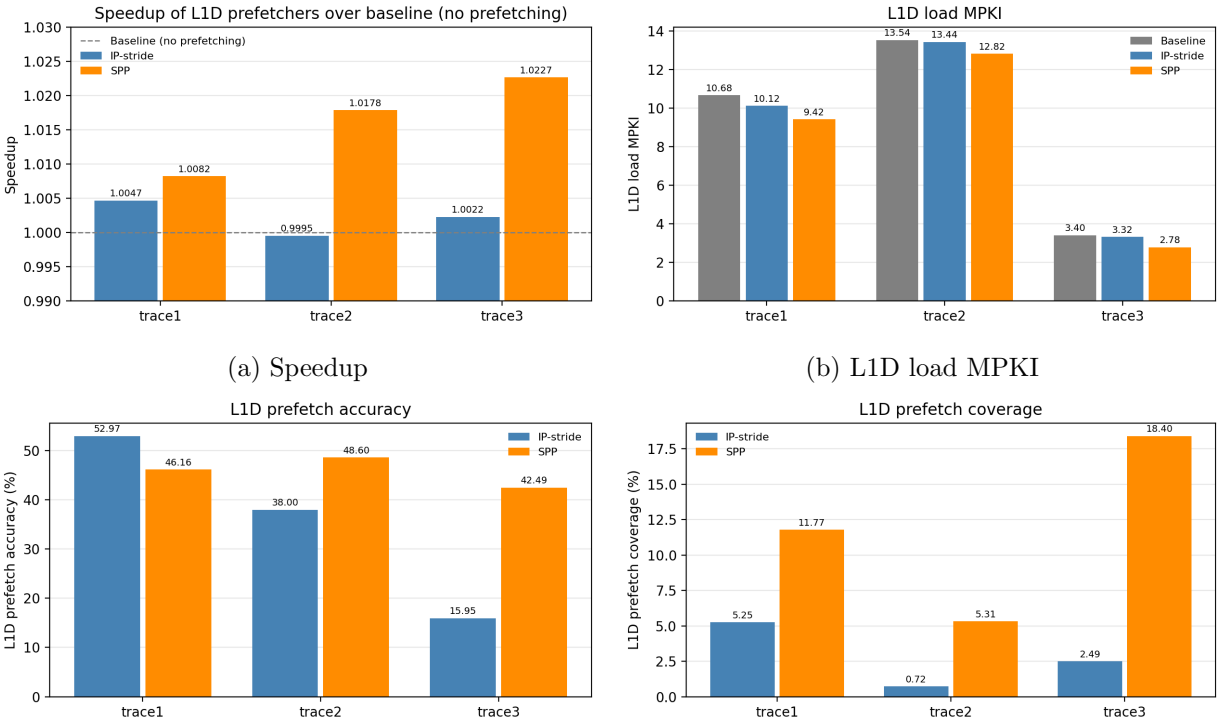
1.2 Results

$$\text{Speedup} = \frac{\text{IPC}_{\text{prefetcher}}}{\text{IPC}_{\text{baseline}}} \qquad \text{Coverage} = \frac{\text{L1D load misses}_{\text{baseline}} - \text{L1D load misses}_{\text{prefetcher}}}{\text{L1D load misses}_{\text{baseline}}}$$

L1D load MPKI and prefetch accuracy are taken from the ChampSim output.

Trace	Config	IPC	Speedup	L1D load MPKI	Accuracy (%)	Coverage (%)
trace1	Baseline	0.2322	1.0000	10.68	–	–
	IP-stride	0.2333	1.0047	10.12	52.97	5.25
	SPP	0.2341	1.0082	9.42	46.16	11.77
trace2	Baseline	0.2513	1.0000	13.54	–	–
	IP-stride	0.2512	0.9995	13.44	38.00	0.72
	SPP	0.2558	1.0178	12.82	48.60	5.31
trace3	Baseline	0.3819	1.0000	3.40	–	–
	IP-stride	0.3827	1.0022	3.32	15.95	2.49
	SPP	0.3905	1.0227	2.78	42.49	18.40
Geomean	IP-stride		1.0021			
	SPP		1.0162			

Table 2: Task 1 results (generated by `scripts/task1/plot_task1.py`).



(a) Speedup

(b) L1D load MPKI

(c) Prefetch accuracy

(d) Prefetch coverage

Figure 1: IP-stride vs SPP.

- SPP is better than IP-stride on all three traces: higher speedup (geometric mean 1.0162 vs 1.0021), lower L1D load MPKI and higher coverage.
- IP-stride slightly slows down trace2 (0.9995).
- On trace1, SPP has lower accuracy than IP-stride (46.16% vs 52.97%) because it prefetches more aggressively to get more coverage (11.77% vs 5.25%). The extra useful prefetches outweigh the wasted ones, so IPC still increases.
- On trace2 and trace3, SPP is also more accurate than IP-stride.

1.3 Storage overhead

Storage is counted as the number of bits each field needs in hardware, not the size of the C++ types used in the simulator (e.g. the stride is stored in an `int64_t` but only needs 7 bits).

Field	Bits	Notes
IP table entry		
IP tag	10	low 10 bits of the IP
Last cache-line address	58	64-bit address − 6 offset bits
Last stride	7	signed, $-63..+63$
Stride confidence	2	
Signature	7	
LRU position	7	$\log_2 128$
Entry total	**91**	
Signature map entry		
Predicted stride	7	signed
Confidence	2	
Entry total	**9**	

Table 3: Bit-level breakdown of one entry.

Structure	Entries	Bits / entry	Total bits
IP table	128	91	11648
Signature map	128	9	1152
Current degree	1	3	3
Epoch fill counter (0–256)	1	9	9
Epoch useful counter (0–256)	1	9	9
Total			**12821 bits ≈ 1.57 KB**

Table 4: Total storage of the SPP.

The last cache-line address (58 of the 91 bits per IP table entry) takes about 58% of the total storage.

1.4 Parameter choices

All speedups are in % over the baseline. Each study changes one parameter and keeps the others fixed; the settings used for the other parameters are given for each study. Accuracy and coverage in these tables are arithmetic means over the three traces.

1.4.1 IP table size

Run with the final values of all other parameters (`run_ip_table.sh`).

Entries (tag bits)	trace1	trace2	trace3	Geomean	Mean acc. (%)	Mean cov. (%)	Storage
24 (8)	0.62	0.79	0.49	0.634	40.5	6.05	0.25 KB
64 (9)	0.76	1.04	2.03	1.275	46.6	10.59	0.70 KB
128 (10)	0.82	1.78	2.27	**1.621**	45.7	11.83	1.42 KB
256 (10)	0.87	1.89	2.46	1.741	45.6	12.68	2.88 KB

Table 5: Speedup for different IP table sizes (% speedup over baseline).

The traces have a large number of load IPs, so a small table loses most IPs before they are trained. Going from 24 to 128 entries gives most of the gain; doubling again to 256 entries adds about 0.1% for twice the storage.

1.4.2 Signature map size

Run with maximum degree 4 and thresholds 0.40 / 0.50 (`run_sig_table.sh`).

Sig. bits (entries)	trace1	trace2	trace3	Geomean	Mean acc. (%)	Mean cov. (%)	Storage
7 (128)	0.84	1.64	2.19	1.556	46.4	11.45	144 B
8 (256)	0.88	1.75	2.22	1.611	47.0	11.97	288 B
9 (512)	0.85	1.77	2.26	1.622	47.4	12.18	576 B
10 (1024)	0.88	1.70	2.27	1.614	47.4	12.24	1152 B

Table 6: Speedup for different signature map sizes (% speedup over baseline).

A larger map gives at most 0.07% more speedup while the map storage grows 2–8 times. We keep 128 entries to keep the prefetcher small.

1.4.3 Confidence

Uniform confidence threshold Run with maximum degree 4 and thresholds 0.40 / 0.50 (`run_uniform_conf.s`

Threshold	trace1	trace2	trace3	Geomean	Mean acc. (%)	Mean cov. (%)
1	0.84	1.64	2.19	**1.556**	46.4	11.45
2	0.79	1.57	2.11	1.488	46.7	10.97
3	0.74	1.57	2.11	1.475	46.4	10.78

Table 7: Speedup for the uniform-mode confidence threshold (% speedup over baseline).

Threshold 1 starts uniform prefetching earlier on constant streams, which raises coverage on all traces while accuracy stays about the same.

1.4.4 Accuracy thresholds

All $(\text{ACC_LOW}, \text{ACC_HIGH})$ pairs on a 0.10 grid with $\text{ACC_HIGH} - \text{ACC_LOW} \geq 0.10$ were simulated with maximum degree 4 (`run_acc_grid.sh`).

ACC_LOW / ACC_HIGH	0.40	0.50	0.60	0.70	0.80
0.10	1.304	1.294	1.349	1.347	1.337
0.20	1.413	1.446	1.471	1.499	1.505
0.30	1.487	1.540	1.551	1.471	1.480
0.40	–	**1.556**	1.520	1.456	1.408
0.50	–	–	1.464	1.431	1.394

Table 8: Geometric mean speedup for each threshold pair (% speedup over baseline).

(0.40, 0.50) has the best speedup in the grid. Below 0.40 most prefetches are wasted, so the degree is lowered; above 0.50 more prefetches are useful than wasted, so the degree is raised. A very low ACC_LOW (0.10) rarely lowers the degree and loses performance.

1.4.5 Degree range

The maximum degree was varied from 2 to 6 at thresholds 0.40 / 0.50; the minimum degree is 1 (`run_max_degree.sh`).

Max degree	trace1	trace2	trace3	Geomean	Mean acc. (%)	Mean cov. (%)
2	0.92	1.17	1.92	1.336	48.4	10.18
3	0.87	1.46	2.10	1.476	47.0	11.01
4	0.84	1.64	2.19	1.556	46.4	11.45
5	0.80	1.72	2.27	1.591	45.9	11.73
6	0.82	1.78	2.27	**1.621**	45.7	11.83

Table 9: Speedup for different maximum degrees (% speedup over baseline).

A higher maximum degree raises coverage and lowers accuracy, and the geomean speedup still increases up to 6. 6 is the largest value tested: the gain from 5 to 6 is only 0.03%, and trace1 already loses speedup at higher degrees (0.92% at 2, 0.82% at 6), so larger values were not pursued. The minimum degree is 1 so that the prefetcher never switches off.

Accuracy	Degree	Reason
> 0.50	+1 (up to 6)	more prefetches used than wasted
0.40–0.50	unchanged	no clear signal, avoids changes on small swings
< 0.40	−1 (down to 1)	most prefetches wasted, reduce pollution and traffic

Table 10: Degree change for each accuracy range.

The degree changes by one step per epoch, so one noisy epoch cannot move it far. It starts at 3 because there is no accuracy information at the beginning. After choosing maximum degree 6, the two best threshold pairs were compared again:

Thresholds	trace1	trace2	trace3	Geomean
(0.30, 0.60)	0.764	1.818	2.219	1.599
(0.40, 0.50)	0.818	1.785	2.266	**1.621**

Table 11: Speedup of the two best threshold pairs at maximum degree 6 (% speedup over baseline).

(0.40, 0.50) is still better at maximum degree 6.

2 Task 2: Instruction-Conscious L2 Replacement

2.1 Task 2A: I-RRIP

Interface change - `update_replacement_state` takes an extra argument `uint8_t isInstruction`. It is added to the function-pointer type and all declarations in `inc/cache.h`, to every replacement policy, and to all call sites in `src/cache.cc` (the packet's `instruction` field) and `src/ooo_cpu.cc` (0 for the BTB).

Policy - I-RRIP (`replacement/irrip.l2c_repl`) is RRIP with 2-bit RRPVs and three changes:

Part	RRIP	I-RRIP
Insertion	all lines at RRPV 2	instruction lines at RRPV 1, data lines at RRPV 2
Promotion	hit → RRPV 0	instruction hit → RRPV 0, data hit: RRPV unchanged
Eviction	first line with RRPV 3	among RRPV 3 lines, a data line before an instruction line

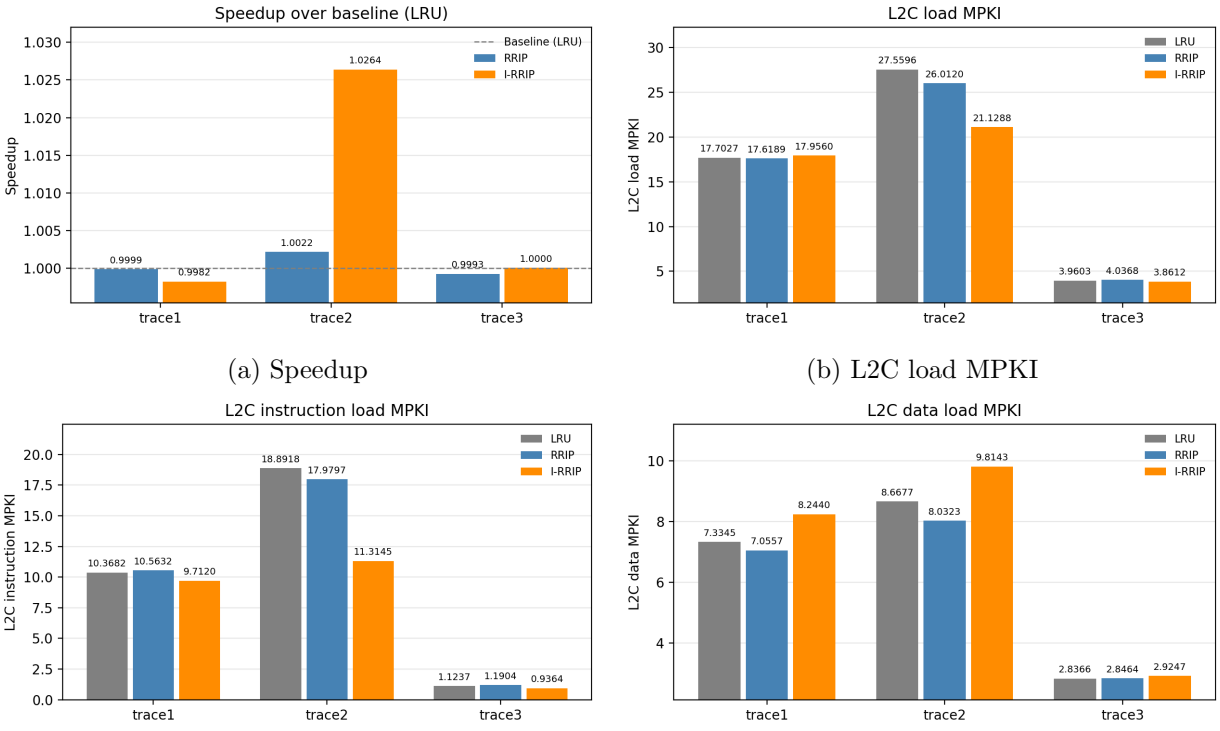
Table 12: RRIP vs I-RRIP.

Results

Speedup is over LRU. All MPKI values are L2C load MPKI from the ChampSim output.

Trace	Policy	IPC	Speedup	L2C MPKI	L2 Instr MPKI	L2 Data MPKI
trace1	LRU	0.2322	1.0000	17.7027	10.3682	7.3345
	RRIP	0.2322	0.9999	17.6189	10.5632	7.0557
	I-RRIP	0.2318	0.9982	17.9560	9.7120	8.2440
trace2	LRU	0.2513	1.0000	27.5596	18.8918	8.6677
	RRIP	0.2518	1.0022	26.0120	17.9797	8.0323
	I-RRIP	0.2579	1.0264	21.1288	11.3145	9.8143
trace3	LRU	0.3819	1.0000	3.9603	1.1237	2.8366
	RRIP	0.3816	0.9993	4.0368	1.1904	2.8464
	I-RRIP	0.3819	1.0000	3.8612	0.9364	2.9247
Geomean	RRIP		1.0004			
	I-RRIP		1.0081			

Table 13: Task 2A results.



(a) Speedup

(b) L2C load MPKI

(c) L2 instruction MPKI

(d) L2 data MPKI

Figure 2: LRU vs RRIP vs I-RRIP.

- I-RRIP reduces L2 instruction MPKI on all three traces (trace2: 18.89 → 11.31, −40%), at the cost of higher data MPKI.
- trace2 has the most instruction misses and gains the most (1.0264); total L2 misses drop by 23%.
- On trace1, the extra data misses are larger than the saved instruction misses, so I-RRIP is slightly slower than LRU (0.9982).

Rationale for insertion, promotion and eviction

All three parts are modified. Insertion and promotion were tested against the alternatives with eviction fixed.

	Speedup				Mean MPKI		
Instr insertion / data promotion	trace1	trace2	trace3	Geomean	L2C load	L2 instr	L2 data
RRPV 1 / not promoted	0.9982	1.0264	1.0000	**1.0081**	14.32	7.32	6.99
RRPV 1 / hit → 0	0.9999	1.0192	0.9996	1.0062	14.81	8.37	6.44
RRPV 1 / hit → 1	0.9998	1.0229	0.9997	1.0074	14.51	7.88	6.64
RRPV 0 / not promoted	0.9971	1.0247	1.0000	1.0072	14.51	7.42	7.09
RRPV 0 / hit → 0	0.9997	1.0202	0.9998	1.0065	14.90	8.27	6.63
RRPV 0 / hit → 1	0.9989	1.0231	0.9998	1.0072	14.64	7.88	6.76

Table 14: Instruction insertion × data promotion; mean MPKI is the arithmetic mean over the three traces.

Insertion -

- RRIP inserts every new line at RRPV 2. I-RRIP inserts instruction lines at RRPV 1, so a new instruction line needs two aging steps before it can be evicted, while a data line needs one.
- Inserting instructions at RRPV 0 is worse in two of the three promotion settings (1.0072 vs 1.0081 without data promotion, 1.0072 vs 1.0074 with data hit → 1). With data hit → 0 it is slightly better (1.0065 vs 1.0062), but both are below the chosen configuration.
- At RRPV 0 every new instruction line is protected, including code that runs only once. These lines push out instruction lines that are reused, so on trace2 even the instruction MPKI goes up (11.31 → 11.63). At RRPV 1 an unused instruction line can still age out.

Promotion -

- An instruction hit promotes the line to RRPV 0 and a data hit does not change the RRPV. Data promotion moves misses between the two types: the less data is promoted, the lower the instruction MPKI and the higher the data MPKI.
- Without data promotion the instruction MPKI is the lowest (mean 7.32, −28% vs LRU), which is the goal of this task, and the total L2 MPKI is also the lowest, because instruction misses are the larger share.
- The cost of not promoting data appears on trace1, whose data lines are reused: it runs at 0.9982 without data promotion and 0.9999 with data hit → 0. The gain on trace2 is much larger, so no data promotion gives the best geomean.

Eviction - Victims are chosen among RRPV 3 lines as in RRIP; on a tie, a data line is evicted before an instruction line to keep instruction lines in the cache.

2.2 Task 2B: Dynamic I-RRIP

Leader sets - 32 of the 1024 L2 sets are leader sets and always run RRIP. The cache is divided into 32 regions of 32 sets with one leader per region: set s is a leader if $s \bmod 32 = \lfloor s/32 \rfloor \bmod 32$ (sets 0, 33, 66, …, 1023). All other sets (followers) run either I-RRIP (as in 2A) or RRIP.

Decision - In the leader sets the policy tracks instruction and non-instruction accesses and misses and uses I-RRIP for the followers when

$$\frac{\text{instr accesses} + \text{instr misses}}{\text{total accesses} + \text{total misses}} > T$$

With I = instr accesses + instr misses and D = data accesses + data misses, this is the same as $(1-T)\,I - T\,D > 0$, so a single counter (PSEL) is used instead of the ratio. With $T = \texttt{NUM}/\texttt{DEN}$, every leader-set access (except writebacks) changes PSEL by

$$\text{instruction: } +\,e\cdot(\texttt{DEN}-\texttt{NUM}) \qquad \text{data: } -\,e\cdot\texttt{NUM} \qquad e = \begin{cases} 1 & \text{hit} \\ 2 & \text{miss} \end{cases}$$

(a miss counts as an access and as a miss). PSEL saturates at its limits; the followers use I-RRIP when PSEL is in its upper half and RRIP otherwise.

Final parameters

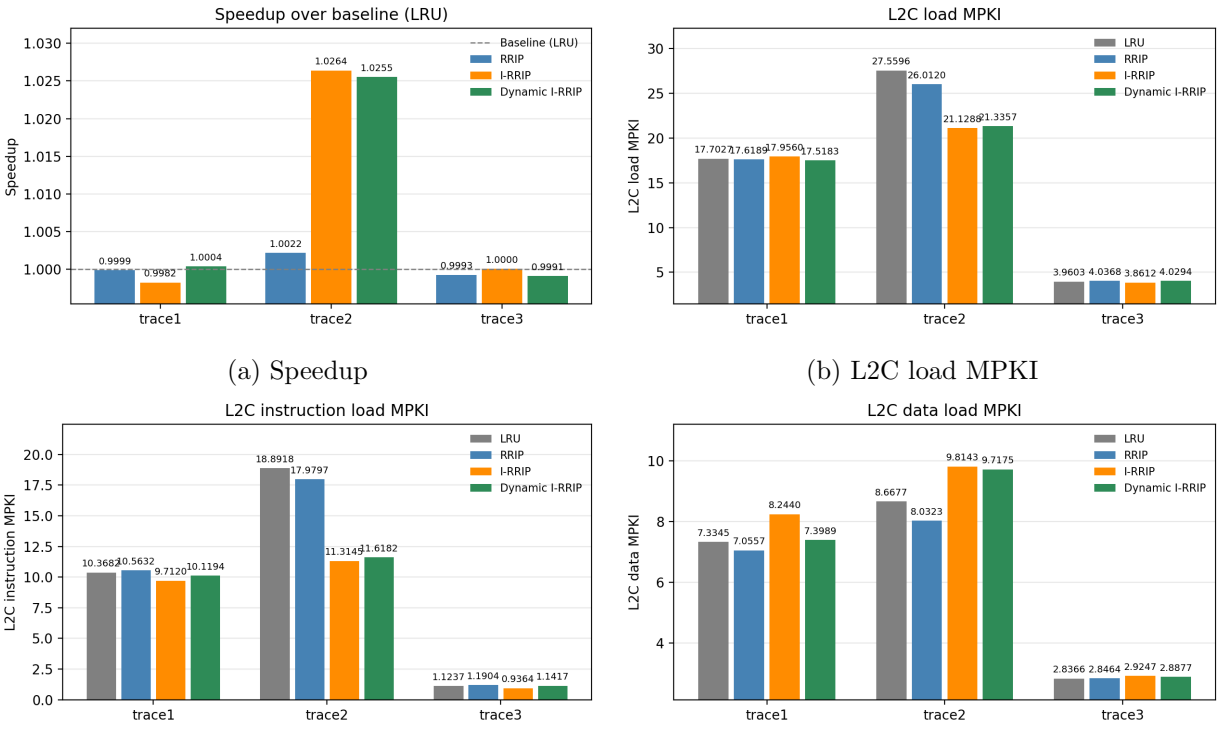
Parameter	Value
Leader sets	32 (one per 32-set region)
Threshold T	0.45 (steps +11 / −9)
PSEL	10 bits, starts at 511, I-RRIP when ≥ 512
I-RRIP mode	as in 2A

Table 15: Final dynamic I-RRIP parameters.

Results

Trace	Policy	IPC	Speedup	L2C MPKI	L2 Instr MPKI	L2 Data MPKI
trace1	LRU	0.2322	1.0000	17.7027	10.3682	7.3345
	RRIP	0.2322	0.9999	17.6189	10.5632	7.0557
	I-RRIP	0.2318	0.9982	17.9560	9.7120	8.2440
	Dynamic I-RRIP	0.2323	1.0004	17.5183	10.1194	7.3989
trace2	LRU	0.2513	1.0000	27.5596	18.8918	8.6677
	RRIP	0.2518	1.0022	26.0120	17.9797	8.0323
	I-RRIP	0.2579	1.0264	21.1288	11.3145	9.8143
	Dynamic I-RRIP	0.2577	1.0255	21.3357	11.6182	9.7175
trace3	LRU	0.3819	1.0000	3.9603	1.1237	2.8366
	RRIP	0.3816	0.9993	4.0368	1.1904	2.8464
	I-RRIP	0.3819	1.0000	3.8612	0.9364	2.9247
	Dynamic I-RRIP	0.3815	0.9991	4.0294	1.1417	2.8877
Geomean	RRIP		1.0004			
	I-RRIP		1.0081			
	Dynamic I-RRIP		1.0082			

Table 16: Task 2B results.



(a) Speedup

(b) L2C load MPKI

(c) L2 instruction MPKI

(d) L2 data MPKI

Figure 3: LRU vs RRIP vs I-RRIP vs dynamic I-RRIP.

- The dynamic policy removes the loss of I-RRIP on trace1 (0.9982 → 1.0004, L2 MPKI 17.96 → 17.52), the trace where prioritizing instructions increases data misses.
- On trace2 it keeps almost all of the I-RRIP gain (1.0255 vs 1.0264); the difference comes from the 32 leader sets, which always run RRIP, and from the followers running RRIP for about 8% of the leader accesses.
- trace3 is data-heavy, so the policy mostly uses RRIP there (0.9991 vs 1.0000 for I-RRIP).
- Geomean speedup is 1.0082 vs 1.0081 for I-RRIP; total L2 misses are slightly lower (arithmetic mean 14.29 vs 14.32), with fewer data misses and more instruction misses.
- Even picking the better of RRIP and I-RRIP for each trace would only give 1.0087, so on these traces there is little left for a dynamic policy to gain over I-RRIP.

Rationale for the number of leader sets and the decision logic

Number of leader sets -

- The leader sets are a sample of the cache: the decision for all follower sets is made from what happens in the leaders.
- More leaders give a more reliable measurement, but every leader set always runs RRIP, even when I-RRIP is better.
- Set dueling (Qureshi et al., *Adaptive Insertion Policies for High Performance Caching*, ISCA 2007) and DRRIP (Jaleel et al., *High Performance Cache Replacement Using Re-Reference Interval Prediction*, ISCA 2010) use 32 leader sets per policy on a cache with 1024 sets, the same as our L2. We use 32 and compare it with 64.

Leader sets	trace1	trace2	trace3	Geomean	Switches (t1 / t2 / t3)
32	1.0004	1.0255	0.9991	**1.0082**	194 / 55 / 17
64	1.0000	1.0248	0.9991	1.0079	352 / 47 / 47

Table 17: Speedup for different numbers of leader sets.

64 leader sets is lower than 32 because twice as many sets are fixed to RRIP.

Decision logic -

- The formula measures how much of the L2 activity in the leader sets comes from instruction lines, counting misses twice.
- This follows the motivation of I-RRIP: when the front end depends heavily on the L2 for instructions (an instruction-heavy phase), instruction lines are protected; otherwise RRIP is used so that data is not hurt.
- The leader sets run plain RRIP so that the measurement is not changed by I-RRIP. If they ran I-RRIP, it would already reduce the instruction misses they count and push the decision back towards RRIP.
- The measured scores (over the whole run) are 0.447 for trace1, 0.62 for trace2 and 0.24 for trace3. A threshold between trace1 and trace2 sends trace2 to I-RRIP, where it gains the most, and keeps trace1 away from static I-RRIP, where it loses.
- Because the counter saturates, old behaviour cannot build up: after a long phase it sits at its limit and needs about half its range of net events to change mode.

PSEL bits	trace1	trace2	trace3	Geomean	Switches (t1 / t2 / t3)
6	0.9993	1.0239	0.9993	1.0074	4201 / 4589 / 457
8	0.9999	1.0255	0.9991	1.0081	921 / 203 / 27
10	1.0004	1.0255	0.9991	**1.0082**	194 / 55 / 17
12	1.0005	1.0253	0.9989	1.0081	95 / 19 / 1

Table 18: Speedup for different PSEL widths.

A narrow counter is noisy: with 6 bits the mode changes more than 4000 times on trace1 and trace2. 10 and 12 bits give the same speedup; 10 bits is the smallest stable width.

Sensitivity to the threshold

T	trace1	trace2	trace3	Geomean	I-RRIP share (%, t1 / t2 / t3)	Switches (t1 / t2 / t3)
0.30	0.9989	1.0254	0.9991	1.0077	95 / 94 / 38	18 / 11 / 3
0.40	0.9993	1.0256	0.9989	1.0078	82 / 95 / 33	54 / 5 / 1
0.45	1.0004	1.0255	0.9991	**1.0082**	51 / 92 / 30	194 / 55 / 17
0.50	0.9999	1.0249	0.9989	1.0078	0 / 94 / 35	0 / 3 / 3
0.60	0.9999	1.0225	0.9990	1.0071	0 / 86 / 20	0 / 27 / 1
0.70	0.9999	1.0055	0.9992	1.0015	0 / 4 / 5	0 / 218 / 1

Table 19: Speedup for different thresholds. I-RRIP share is the percentage of leader-set accesses during which the followers ran I-RRIP.

- Below 0.447 (trace1's score), trace1 mostly runs I-RRIP and loses slightly (0.9989, 0.9993).

- At 0.50 trace1 runs only RRIP and trace2 mostly I-RRIP.
- At 0.45 trace1's score is at the threshold, so it uses each mode about half of the time; its L2 MPKI (17.52) is lower than with either policy alone (RRIP 17.62, I-RRIP 17.96). This gives the best geomean, but it depends on trace1's score being almost exactly 0.45, so it is specific to these traces. The gain over 0.50 is only 0.04%.
- At 0.60 trace2 is close to the threshold and loses part of its gain; at 0.70 it mostly runs RRIP and the gain is gone (1.0055).
- The example threshold 0.25 from the assignment is below all three scores on these traces, so the followers would almost always run I-RRIP.

3 Task 3: SPP with Dynamic I-RRIP

3.1 Design

Setup - SPP runs at L1D and the Dynamic I-RRIP policy runs at L2. When an SPP prefetch misses in L1D, the request is sent to L2 with type `PREFETCH` and the returning line is filled into L2 as well as L1D. The replacement policy (`replacement/dyn_irrip.l2c_repl`) uses the `type` argument of `l2c_update_replacement_state` to detect these accesses.

Changes to Dynamic I-RRIP -

Access	Dynamic I-RRIP (Task 2B)	Prefetch-aware Dynamic I-RRIP
Prefetch fill	treated as a data fill	RRPV 3
Prefetch hit	treated as a data hit	RRPV unchanged
Demand hit on a prefetched line	same as any demand hit	same as any demand hit
Leader-set sampling	all accesses except writebacks	prefetch fills and hits are not sampled

Table 20: Handling of prefetch accesses at L2.

- Insertion, promotion and eviction of demand accesses are unchanged from Task 2B.
- If a demand load arrives while a prefetch for the same line is still in the MSHR, ChampSim changes the MSHR type to the demand type. That fill is then handled as a normal demand fill and is sampled.

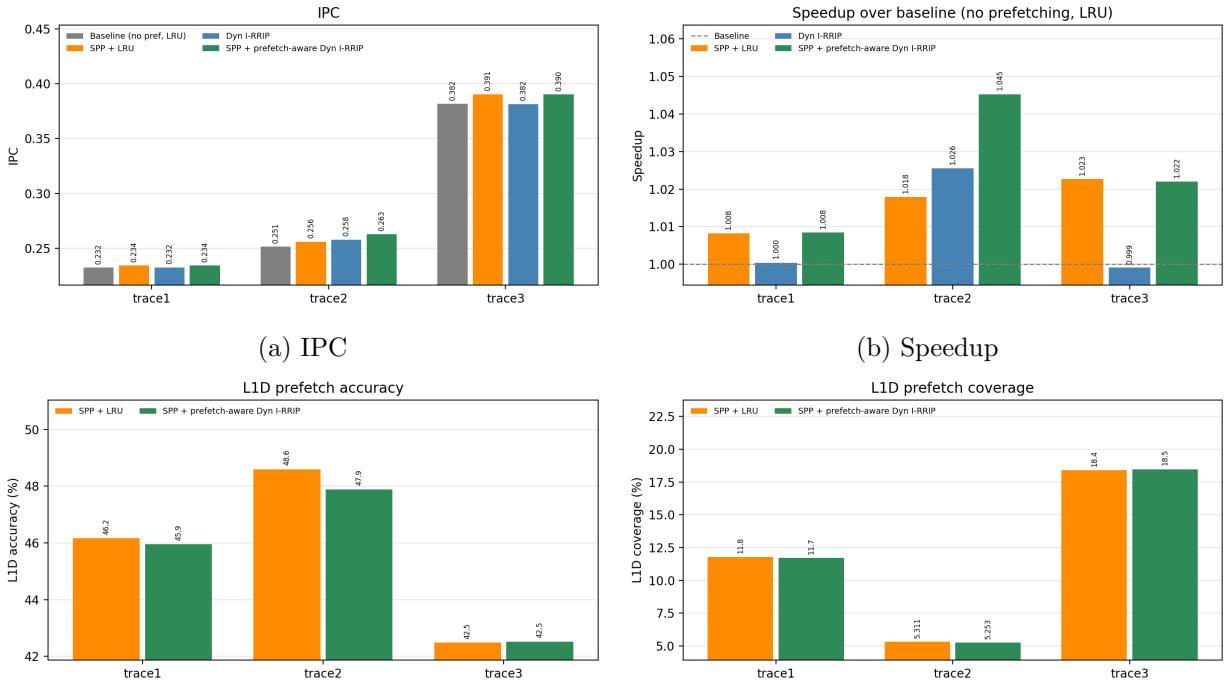
3.2 Results

Speedup and coverage are computed as in Task 1, with the baseline being no prefetcher and LRU at L2. The four configurations are run by `scripts/task3/run_3.sh`:

- **Baseline**: no prefetcher, LRU at L2.
- **SPP**: SPP at L1D, LRU at L2.
- **Dyn I-RRIP**: no prefetcher, Dynamic I-RRIP at L2.
- **SPP + Dyn I-RRIP**: SPP at L1D, prefetch-aware Dynamic I-RRIP at L2.

Trace	Config	IPC	Speedup	L1D load MPKI	Accuracy (%)	Coverage (%)	L2C load MPKI	L2C instr. MPKI	L2C data MPKI
trace1	Baseline	0.2322	1.0000	10.68	–	–	17.70	10.37	7.33
	SPP	0.2341	1.0082	9.42	46.16	11.77	16.69	10.43	6.27
	Dyn I-RRIP	0.2323	1.0004	10.68	–	–	17.52	10.12	7.40
	SPP + Dyn I-RRIP	0.2342	1.0084	9.43	45.95	11.71	16.37	9.97	6.40
trace2	Baseline	0.2513	1.0000	13.54	–	–	27.56	18.89	8.67
	SPP	0.2558	1.0178	12.82	48.60	5.31	27.12	19.17	7.96
	Dyn I-RRIP	0.2577	1.0255	13.54	–	–	21.34	11.62	9.72
	SPP + Dyn I-RRIP	0.2627	1.0453	12.83	47.89	5.25	20.70	11.69	9.01
trace3	Baseline	0.3819	1.0000	3.40	–	–	3.96	1.12	2.84
	SPP	0.3905	1.0227	2.78	42.49	18.40	3.18	1.14	2.04
	Dyn I-RRIP	0.3815	0.9991	3.40	–	–	4.03	1.14	2.89
	SPP + Dyn I-RRIP	0.3903	1.0220	2.77	42.52	18.48	3.23	1.14	2.09
Geomean	SPP	0.2860	1.0162						
	Dyn I-RRIP	0.2837	1.0082						
	SPP + Dyn I-RRIP	0.2885	**1.0251**						

Table 21: Task 3 results



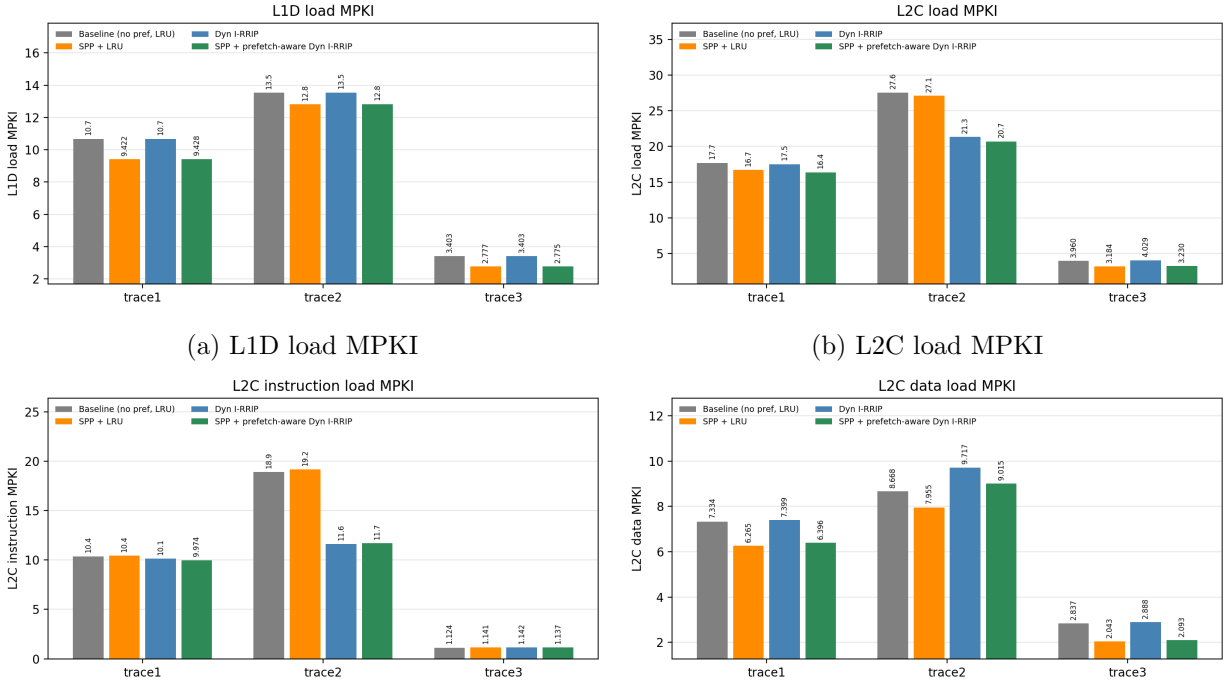
(a) IPC

(b) Speedup

(c) L1D prefetch accuracy

(d) L1D prefetch coverage

Figure 4: IPC, speedup, accuracy and coverage.



(a) L1D load MPKI

(b) L2C load MPKI

(c) L2C instruction load MPKI

(d) L2C data load MPKI

Figure 5: Load MPKI at L1D and L2C.

- SPP + Dyn I-RRIP has the best geometric mean speedup: 1.0251, compared to 1.0162 for SPP and 1.0082 for Dyn I-RRIP.
- The two techniques mostly help different misses: SPP lowers L1D load MPKI, and Dyn I-RRIP lowers L2C instruction MPKI. The combination keeps both reductions, e.g. on trace2 L1D load MPKI goes from 13.54 to 12.83 and L2C instruction MPKI from 18.89 to 11.69, giving 1.0453.
- L1D accuracy and coverage are almost the same as standalone SPP, since the L2 policy does not change what SPP prefetches.
- On trace1 the combination is slightly better than SPP (1.0084 vs 1.0082).
- On trace3 the combination is 0.07% slower than SPP (1.0220 vs 1.0227). Trace3 has very few L2 instruction misses (MPKI $\approx$ 1.1), so I-RRIP mode has little to gain, and the followers run RRIP most of the time, which is 0.07% slower than LRU on this trace (Task 2A). SPP accuracy and coverage are unchanged, so the gap is not caused by the prefetch handling.

3.3 Rationale for handling prefetch packets

Prefetch fill at RRPV 3 -

- A prefetched line is a prediction; a demand line is known to be needed. If an SPP prefetch is useful, the demand load hits in L1D, so the L2 copy of the line is usually not accessed again. If the prefetch is useless, the L2 copy is never used. Either way, the L2 copy has little reuse.
- The ChampSim stats confirm this: with SPP + LRU, only 5–6% of the prefetched lines in L2 are later used by a demand access (trace1: 4268 useful, 68123 useless).
- Inserting at RRPV 3 makes these lines the first victims, so they do not push out instruction lines, which Dyn I-RRIP is trying to keep in L2.
- A prefetched line that does get a demand hit at L2 is promoted as usual, so a useful line is still protected after its first real use.

No promotion on prefetch hit -

- A prefetch hit happens when SPP requests a line that is already in L2. Plain RRIP would promote this line to RRPV 0 as if the program had reused it.
- The access comes from the prefetcher, not from the program, so it is not considered as reuse. The RRPV is left unchanged.

Prefetches not sampled by leader sets -

- The decision formula (instr. accesses + instr. misses)/(total accesses + total misses) should measure how much the program needs instruction lines in L2.
- SPP prefetches are all data accesses. Counting them adds data events to the denominator and moves the policy selector towards RRIP.
- The mode would then depend on how aggressive SPP is. Prefetch fills and hits are therefore not sampled.

Effect on L2 prefetch traffic - Prefetches are only 2.6–13% of L2 accesses (e.g. trace1: 102597 of 1072109), so the prefetch handling mainly protects the existing L2 contents; most of the gain comes from combining the L1D and L2 improvements.